

Course: Full Stack Web Development	Course Code: 23CM4121
Experiment No.: 2	Date: 05-09-2026
Student Name: B. Venkata sai laxmi	Roll No.: A24126552074

1. TITLE :

Develop React Functional and Class Components using State, Props and Lifecycle Methods.

2. AIM :

To develop a React application that demonstrates functional components (using Hooks) and class components (using lifecycle methods), and to compare how each manages state, props and the mounting, updating and unmounting phases.

3. OBJECTIVE :

The objectives of this experiment are:

- To create a React application composed of reusable components (App, FunctionalCounter and ClassCounter).
- To implement a functional component that manages state with the useState hook.
- To implement a class component that manages state with this.state and this.setState().
- To pass data into components using props (initialValue and label).
- To observe lifecycle behaviour using useEffect in the functional component and componentDidMount, componentDidUpdate and componentWillUnmount in the class component.
- To demonstrate conditional rendering, which mounts and unmounts components on a button click.
- To handle user events (onClick) and render the application into the DOM using ReactDOM.createRoot().

4. THEORY :

React is a JavaScript library used to build user interfaces from small, reusable pieces called components. Each component describes what should appear on screen for a given state and props, and React updates the DOM efficiently when they change.

Components

A component is a self-contained unit of UI. React supports two ways of writing components:

- **Functional components:** plain JavaScript functions that accept props and return JSX. State and lifecycle features are added through Hooks such as useState and useEffect.
- **Class components:** ES6 classes that extend React.Component. They keep state in this.state, update it with this.setState(), and use lifecycle methods and a render() method.

JSX

JSX is a syntax extension that lets HTML-like markup be written inside JavaScript. It is compiled to React.createElement() calls. JavaScript expressions are embedded using curly braces { }.

Props and State

- **Props** are read-only inputs passed from a parent to a child component (for example initialValue and label).

- **State** is data owned by a component that can change over time. Changing state triggers a re-render.

Hooks used

- **useState(initial):** returns the current state value and a setter function.
- **useEffect(callback, deps):** runs side effects after render. An empty dependency array [] runs it once on mount, and the returned cleanup function runs on unmount. A dependency such as [count] runs it whenever count changes.

Component Lifecycle

Every component passes through three phases: Mounting (created and inserted into the DOM), Updating (re-rendered when state or props change) and Unmounting (removed from the DOM).

Phase	Class component	Functional component
Mounting	componentDidMount()	useEffect(() => {...}, [])
Updating	componentDidUpdate(prevProps, prevState)	useEffect(() => {...}, [count])
Unmounting	componentWillUnmount()	cleanup function returned from useEffect

Functional vs Class Components

Feature	Functional component	Class component
Definition	JavaScript function	ES6 class extending Component
State	useState hook	this.state and this.setState()
Lifecycle	useEffect hook	Lifecycle methods
Props access	Function parameter	this.props
Syntax	Shorter, no "this" keyword	Longer, requires constructor, render() and "this"

Conditional Rendering and Events

Using the expression {condition && <Component />}, React renders a component only when the condition is true. Toggling the condition mounts or unmounts the component. Events such as onClick are written in camelCase and receive a function as the handler.

Flow of the Application

index.js → ReactDOM.createRoot() → renders <App /> → App renders FunctionalCounter and ClassCounter → user clicks a button → state changes → component re-renders → lifecycle logs appear in the console.

5. ALGORITHM / PROCESS :

1. Create a new React project (for example using Vite or Create React App) and open it in a code editor.
2. Inside the src folder create four files: FunctionalCounter.js, ClassCounter.js, App.js and index.js.
3. In FunctionalCounter.js, import React, useState and useEffect.

4. Define the function `FunctionalCounter` that receives the props `initialValue` and `label`.
5. Declare state with `const [count, setCount] = useState(initialValue)`.
6. Add a `useEffect` with the dependency `[count]` that logs the new count whenever it changes.
7. Add a `useEffect` with an empty dependency array that logs "mounted" and returns a cleanup function that logs "unmounted".
8. Create the increment, decrement and reset functions using `setCount()`.
9. Return JSX containing a heading, the current count and three buttons, and export the component.
10. In `ClassCounter.js`, import `React` and `Component`, then define class `ClassCounter` extends `Component`.
11. In the constructor, call `super(props)` and initialise `this.state = { count: props.initialValue || 0 }`.
12. Implement `componentDidMount()`, `componentDidUpdate(prevProps, prevState)` and `componentWillUnmount()` to log lifecycle messages.
13. Create increment, decrement and reset as arrow-function class properties that call `this.setState()`.
14. Implement `render()` to return JSX with the heading, count and buttons, and export the class.
15. In `App.js`, import `useState` and both counter components.
16. Create the state `showComponents = true` and a toggle button labelled "Unmount Components" or "Mount Components".
17. Conditionally render `<FunctionalCounter initialValue={10} label="Hooks" />` and `<ClassCounter initialValue={20} label="Lifecycle" />` inside a fragment.
18. In `index.js`, get the root element, call `ReactDOM.createRoot()` and render `<App />`.
19. Start the development server (`npm start` or `npm run dev`) and open the application in the browser.
20. Open the browser console (F12) to observe the lifecycle messages.
21. Click Increment, Decrement and Reset on each counter and verify the count and console output.
22. Click Unmount Components and then Mount Components to verify the unmount and mount lifecycle logs.

6. PROGRAM :

FunctionalCounter.js

```
import React, { useState, useEffect } from 'react';

function FunctionalCounter({ initialValue = 0, label }) {
  // State using useState hook
  const [count, setCount] = useState(initialValue);

  // Lifecycle: runs whenever count changes (like componentDidUpdate)
  useEffect(() => {
    console.log(`[Functional] Count changed to: ${count}`);
  }, [count]);

  // Lifecycle: runs once on mount (like componentDidMount)
  useEffect(() => {
    console.log('[Functional] Component mounted');
    return () => console.log('[Functional] Component unmounted');
  }, []);

  const increment = () => setCount(count + 1);
  const decrement = () => setCount(count - 1);
  const reset = () => setCount(initialValue);
```

```

return (
  <div style={{ border: '2px solid green', padding: '16px', margin: '10px' }}>
    <h2>Functional Component {label} && '- ${label}`</h2>
    <p>Count: <strong>{count}</strong></p>
    <button onClick={increment}>+ Increment</button>{' '}
    <button onClick={decrement}>- Decrement</button>{' '}
    <button onClick={reset}>Reset</button>
  </div>
);
}

export default FunctionalCounter;

```

ClassCounter.js

```

import React, { Component } from 'react';

class ClassCounter extends Component {
  // Constructor to initialize state
  constructor(props) {
    super(props);
    this.state = {
      count: props.initialValue || 0,
    };
  }

  // Lifecycle: runs once after component is mounted
  componentDidMount() {
    console.log('[Class] Component mounted');
  }

  // Lifecycle: runs after every state/prop update
  componentDidUpdate(prevProps, prevState) {
    if (prevState.count !== this.state.count) {
      console.log(`[Class] Count changed to: ${this.state.count}`);
    }
  }

  // Lifecycle: runs before component is removed
  componentWillUnmount() {
    console.log('[Class] Component unmounted');
  }

  increment = () => this.setState({ count: this.state.count + 1 });
  decrement = () => this.setState({ count: this.state.count - 1 });
  reset = () => this.setState({ count: this.props.initialValue || 0 });

  render() {
    const { label } = this.props;
    const { count } = this.state;

    return (
      <div style={{ border: '2px solid blue', padding: '16px', margin: '10px' }}>
        <h2>Class Component {label} && '- ${label}`</h2>
        <p>Count: <strong>{count}</strong></p>
        <button onClick={this.increment}>+ Increment</button>{' '}
        <button onClick={this.decrement}>- Decrement</button>{' '}
        <button onClick={this.reset}>Reset</button>
      </div>
    );
  }
}

export default ClassCounter;

```

App.js

```
import React, { useState } from 'react';
import FunctionalCounter from './FunctionalCounter';
import ClassCounter from './ClassCounter';

function App() {
  const [showComponents, setShowComponents] = useState(true);

  return (
    <div style={{ fontFamily: 'Arial', padding: '20px' }}>
      <h1>React Components Demo</h1>

      <button onClick={() => setShowComponents(!showComponents)}>
        {showComponents ? 'Unmount Components' : 'Mount Components'}
      </button>

      {showComponents && (
        <>
          <FunctionalCounter initialValue={10} label="Hooks" />
          <ClassCounter initialValue={20} label="Lifecycle" />
        </>
      )}
    </div>
  );
}

export default App;
```

index.js

```
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

7. CODE EXPLANATION :

Code	Explanation
import React, { useState, useEffect }	Imports React and the two Hooks needed by the functional component.
function FunctionalCounter({ initialValue = 0, label })	Functional component. Props are destructured and initialValue defaults to 0.
const [count, setCount] = useState(initialValue)	Creates the state variable count and its setter setCount.
useEffect(() => {...}, [count])	Runs after every render in which count has changed and logs the new value (update phase).
useEffect(() => {...}, [])	Runs once after the first render (mount). The returned function runs when the component is removed (unmount).
increment / decrement / reset	Event handlers that update the state with setCount(); reset restores initialValue.
class ClassCounter extends Component	Defines a class component by extending React.Component.

Code	Explanation
<code>constructor(props) { super(props); this.state = {...} }</code>	Initialises the state object. <code>super(props)</code> is required before using this.
<code>componentDidMount()</code>	Called once after the component is inserted into the DOM.
<code>componentDidUpdate(prevProps, prevState)</code>	Called after each update. The if-condition logs only when count has changed, which avoids unnecessary logs.
<code>componentWillUnmount()</code>	Called just before the component is removed from the DOM; used for cleanup.
<code>increment = () => this.setState({...})</code>	Arrow-function class properties keep "this" bound to the component. <code>setState()</code> updates the state and triggers a re-render.
<code>render()</code>	Returns the JSX for the class component. Data is read from <code>this.props</code> and <code>this.state</code> .
<code>style={{ border: ..., padding: ... }}</code>	Inline styles passed as a JavaScript object (outer braces for the expression, inner braces for the object).
<code>{label && `-\${label}`}</code>	Conditional expression that shows the label only when the prop is provided.
<code>const [showComponents, setShowComponents] = useState(true)</code>	State in App that decides whether the counters are displayed.
<code>{showComponents && (<...>/>)}</code>	Conditional rendering. The fragment <code><> </></code> groups both components without adding an extra DOM element.
<code>initialValue={10} label="Hooks"</code>	Props passed from the parent App to the child components.
<code>ReactDOM.createRoot(document.getElementById("root"))</code>	Creates the React root attached to the div with id "root" in index.html.
<code>root.render(<App />)</code>	Renders the App component, and therefore the whole component tree, into the root.
<code>export default / import</code>	ES module syntax used to share components between files.

8. TESTCASES AND EXECUTION DATA:

Test Case No.	Test Case Description	Input Data	Expected Output	Actual Output	Status
1	Verify application loads	Open the app in browser	Heading, toggle button and both counters are displayed	Heading, toggle button and both counters are displayed	PASS
2	Verify initial values passed through props	View counters on first load	Functional count = 10, Class count = 20	Functional count = 10, Class count = 20	PASS
3	Increment functional counter	Click "+ Increment" on Functional component	Count becomes 11	Count becomes 11	PASS
4	Decrement functional counter	Click "- Decrement" on Functional component	Count decreases by 1	Count decreases by 1	PASS
5	Reset functional counter	Change the count, then click "Reset"	Count returns to 10	Count returns to 10	PASS
6	Increment class counter	Click "+ Increment" on Class component	Count becomes 21	Count becomes 21	PASS
7	Decrement class counter	Click "- Decrement" on Class component	Count decreases by 1	Count decreases by 1	PASS
8	Reset class counter	Change the count, then click "Reset"	Count returns to 20	Count returns to 20	PASS
9	Verify independent state	Increment only the functional counter	Class counter value is unchanged	Class counter value is unchanged	PASS
10	Verify mount logs	Reload page and open console	"[Functional] Component mounted" and "[Class] Component mounted" are logged	Both mount messages are logged	PASS
11	Verify update logs	Click Increment on each counter	"Count changed to: n" is logged for the counter that changed	Update message is logged for the changed counter	PASS
12	Verify unmounting	Click "Unmount Components"	Both counters disappear, button text becomes "Mount Components", unmount messages are logged	Both counters removed, button text changed, unmount messages logged	PASS
13	Verify remounting	Click "Mount Components"	Counters reappear with fresh state (10 and 20) and mount logs appear	Counters reappear with 10 and 20, mount logs shown	PASS

9. OUTPUT:

a) Initial render of the application (Functional = 10, Class = 20):

React Components Demo

Unmount Components

Functional Component - Hooks
Count: 10

+ Increment - Decrement Reset

Class Component - Lifecycle
Count: 20

+ Increment - Decrement Reset

b) After clicking Increment / Decrement:

React Components Demo

Unmount Components

Functional Component - Hooks
Count: 12

+ Increment - Decrement Reset

Class Component - Lifecycle
Count: 22

+ Increment - Decrement Reset

c) After clicking Reset on both counters:

React Components Demo

Unmount Components

Functional Component - Hooks

Count: 10

+ Increment

- Decrement

Reset

Class Component - Lifecycle

Count: 20

+ Increment

- Decrement

Reset

d) After clicking "Unmount Components":

React Components Demo

Mount Components

e) After clicking "Mount Components" again (state re-initialised):

React Components Demo

Unmount Components

Functional Component - Hooks
Count: 10

+ Increment - Decrement Reset

Class Component - Lifecycle
Count: 20

+ Increment - Decrement Reset

f) Browser console showing lifecycle messages:

Console top Default levels
[Class] Component mounted
[Functional] Count changed to: 10
[Functional] Component mounted
[Functional] Count changed to: 11
[Functional] Count changed to: 12
[Functional] Count changed to: 13
[Functional] Count changed to: 12
[Class] Count changed to: 21
[Class] Count changed to: 22
[Functional] Count changed to: 10
[Class] Count changed to: 20
[Class] Component unmounted
[Functional] Component unmounted
[Class] Component mounted
[Functional] Count changed to: 10
[Functional] Component mounted

10. RESULT:

The React application was successfully developed using both a functional component and a class component. The functional component managed its state with the `useState` hook and handled lifecycle events with `useEffect`, while the class component used `this.state`, `this.setState()` and the lifecycle methods `componentDidMount`, `componentDidUpdate` and `componentWillUnmount`. Props passed from the App component initialised each counter correctly, and the increment, decrement and reset operations updated the state independently. Conditional rendering mounted and unmounted the components, and the corresponding lifecycle messages appeared in the browser console. All test cases passed, confirming the equivalent behaviour of functional and class components in React.